AWS Infrastructure Automation with Terraform Project Summary

Project Overview

Project Objectives

- Automate AWS infrastructure deployment using Terraform
- Create secure, scalable architecture for two websites
- Implement Infrastructure as Code (IaC) best practices
- Demonstrate end-to-end cloud engineering skills

Architecture Components

Networking Layer

- **VPC**: 172.16.0.0/16 with DNS support enabled
- **Subnets**: 6 total (2 public, 2 private-web, 2 private-data)
- **Internet Gateway**: For public subnet internet access
- **NAT Gateway**: For private subnet outbound connectivity
- **Route Tables**: Separate public and private routing

Security Layer

- **Security Groups**: Layer-based security (ALB, Web, Database)
- **Network ACLs**: Subnet-level traffic filtering
- **IAM Roles**: Least privilege access for EC2 instances

Compute Layer

- **EC2 Instances**: 4 Ubuntu 22.04 instances (2 per website)
- **Instance Type**: t3.micro with auto-scaling capability
- **User Data**: Automated Nginx installation and configuration

Load Balancing

- **Application Load Balancers**: 2 ALBs (one per website)
- **Target Groups**: Health-checked endpoints with proper routing
- **Listeners**: HTTP traffic distribution

Database Layer

- **RDS MySQL**: 2 managed database instances
- **Multi-AZ**: High availability configuration

- **Encryption**: Storage and transit encryption enabled

Key Features Implemented

- **Infrastructure as Code**: Complete environment defined in Terraform
- **Security Best Practices**: Principle of least privilege applied
- **High Availability**: Multi-AZ deployment across availability zones
- **Automated Deployment**: Single command deployment (terraform apply)
- **Cost Optimization**: t3.micro instances, proper resource tagging
- **State Management**: Remote state configuration (S3 backend)
- **Tagging Strategy**: Consistent resource tagging for cost tracking

Learning Outcomes

Technical Skills Developed

- Terraform configuration and module creation
- AWS networking (VPC, subnets, routing)
- Security group design and implementation
- Load balancer configuration and health checks
- Database deployment and management
- IAM role and policy creation

DevOps Practices Mastered

- Infrastructure as Code principles
- Version control for infrastructure
- Environment parity (dev/prod consistency)
- Automated testing and validation
- Documentation and knowledge sharing

Challenges Overcome

1. **AMI Compatibility**: Resolved region-specific AMI availability
2. **SSH Key Management**: Fixed file encoding and path issues
3. **RDS Naming Constraints**: Adapted to AWS naming requirements
4. **Security Implementation**: Balanced accessibility with security
5. **State Management**: Configured remote state storage

Business Value Delivered

- **Time Savings**: Reduced deployment time from hours to minutes
- **Cost Efficiency**: Proper resource sizing and auto-scaling
- **Reliability**: Repeatable, consistent deployments
- **Security**: Built-in security controls and monitoring
- **Scalability**: Architecture designed for future growth

Project Links

GitHub Repository:
StegHub_DevOps/project15 at main · 123Origami/StegHub_DevOps

LinkedIn Profile:
https://www.linkedin.com/posts/sally-m-09546a245_minecraft-terraform-aws-activity-7410757187825864704-a8Gv?utm_source=share&utm_medium=member_desktop&rcm=ACoAADzCKfEBZK3LPTQ5D0pi879scjlAX0g4FTk

Contact Information

Name: Sally Mukami Munga

This project demonstrates professional cloud engineering capabilities using industry-standard tools and following AWS best practices. The infrastructure was successfully deployed, tested, and destroyed following proper DevOps lifecycle management.